

OMNIVERSAL ARCHITECT LABS

OAL AI UNIFIED

Adversarial Cross-Review, Executable
Validation,
and Iterative Improvement Protocol

*A reusable non-consensus process for turning multi-AI documentation into testable,
reproducible, versioned evidence*

Field	Entry
Document ID	OAL-AIU-ARVIP-CHATGPT-001
Version	v0.1 (initial independent protocol render)
Date	October 1, 2026
Prepared by	ChatGPT (OpenAI), independent AI collaboration member
Prepared for	Nathan Lewis II, Architect of Record, Omniversal Architect Labs
Status	PROPOSED. NOT CANON. NOT ONESTAMP-RATIFIED. PENDING ARCHITECT REVIEW.
Scope	OAL AI Unified; adaptable to One Industries and Lewis Corp by project-specific prefix and source rules
Posture	Adversarial, non-consensus, execution-oriented, provenance-preserving

Contents

Section	Title
1	Executive Summary
2	Mission Statement and Directive
3	Overview: What This Protocol Changes
4	Source Basis and Lessons From the End Cycle Cross-Review
5	Independent Feedback, Criticism, Questions, and Concerns
6	Core Design Principles
7	Status, Evidence Maturity, and Epistemic Labels
8	When to Run the Process: Milestone Triggers and Resource Modes
9	The Full Adversarial Review and Validation Lifecycle
10	Gate 0 - Commission, Scope, and Corpus Freeze
11	Gate 1 - Independent Adversarial Review
12	Gate 2 - Discrepancy, Contradiction, and Open-Work Registers
13	Gate 3 - Test Conversion and Pre-registration
14	Gate 4 - Execution Standards
15	Gate 5 - Replication, Blinding, and Independent Judgment
16	Gate 6 - Evidence Synthesis Without Consensus Laundering
17	Gate 7 - Iteration Commission and Change Control
18	Gate 8 - Publication, GitHub Packaging, and Reproducibility
19	Minimum Common Templates
20	Copy-Ready Cross-Topic Master Commission Prompt
21	Quality Gates and Automatic Review Failures
22	Adversarial Failure-Mode Catalogue
23	How to Apply the Protocol to Different OAL Topics
24	Case Study: End Cycle Laboratory Reproducibility Check
25	Questions Reserved for the Architect
26	Recommended Adoption Path
Appendix A	Reusable Forms and Tables
Appendix B	Proposed GitHub Directory Structure

Section	Title
Appendix C	Proposed Artifact and ID Codes
Appendix D	Run Manifest / Reproducibility Record
Appendix E	Source Register and Provenance Notes

1. Executive Summary

The most important lesson from the End Cycle Laboratory cross-review is not that one collaborator produced the "correct" interpretation. The advancement is methodological: a large body of polished design material was forced to answer a simpler question - what was actually executed? The answer was that the Iteration 1 documents contained extensive specification but no exercise results. Perplexity Computer then ran three small simulations, published the assumptions, code, seed, and raw results, and explicitly limited the findings to the models. That move changed the work from a discussion about what a laboratory should be into an example of a laboratory producing evidence that could be criticized, rerun, and rejected.

This protocol generalizes that move across OAL AI Unified topics. It creates a repeatable sequence for source freezing, independent review, adversarial discrepancy analysis, test conversion, pre-registration, execution, replication, synthesis, iteration, and publication. It is intentionally designed so that disagreement can remain unresolved. The protocol does not treat agreement among AI systems as proof, document length as progress, or a detailed framework as evidence that the framework works.

The proposed operating model is milestone-driven rather than calendar-driven. A full six-system cross-review does not need to be run every month. Routine documentation can continue normally. The heavier process is triggered when a topic reaches a meaningful decision point: before canonization or ONESTAMP consideration, after a major redesign, when versions materially conflict, when a framework claims to be executable, when a result is being generalized beyond a toy model, or when a high-impact architectural decision depends on it.

The protocol in one view

Gate	Primary action	What changes
0. Commission / freeze	Lock scope, source packet, versions, missing inputs, and status.	A reviewable corpus exists.
1. Independent adversarial review	Each reviewer identifies contributions, defects, assumptions, contradictions, missing work, and executable claims.	No consensus synthesis yet.
2. Registers	Create discrepancy, claim, open-question, and unperformed-task registers.	Nothing important is hidden in prose.
3. Pre-register	Convert selected claims into hypotheses, falsifiers, controls, parameters, and stop rules before the run.	The test can fail.
4. Execute	Run the smallest useful exercises; preserve code, prompts, raw outputs, failures, and environment.	Specification becomes evidence.
5. Replicate / blind	Rerun exactly, vary seeds/parameters, reimplement where possible, and separate generator from evaluator.	Circularity is reduced.

Gate	Primary action	What changes
6. Synthesize	Compare results and disagreements without averaging them away.	A result map replaces a consensus verdict.
7. Iterate	Issue a change ledger and a new commission tied to specific evidence and unresolved discrepancies.	Iteration has a reason.
8. Publish	Package documents, manifests, hashes, code, data, and known limitations for GitHub.	History can be audited later.

Independent recommendation

Make execution a maturity threshold, not a universal burden

Do not require every early concept note to run code. Instead, prohibit a document from claiming FINDINGS until something has been executed. A topic may remain intentionally at SPECIFICATION status for months. The expensive work should be concentrated on the claims that survive adversarial review and matter enough to influence the next design decision.

2. Mission Statement and Directive

Mission statement

Proposed mission

The OAL AI Unified Adversarial Review and Validation Protocol exists to make documentation answerable to execution. It converts important claims into tests that can fail; separates source material, interpretation, evidence, and proposal; exposes contradictions and unfinished work; preserves independent dissent; and carries only the results that survive reproducibility and review into the next iteration.

Directive

- Challenge the work before extending it.
- Run the smallest test that could disprove a claim before building the largest structure that assumes it.
- Never treat an unexecuted exercise, predicted outcome, or agreed-upon narrative as a finding.
- Preserve the original independent versions before collaborators see one another's work.
- Record missing sources, unrun tests, failed tests, contradictory results, and unresolved questions as first-class outputs.
- Make every computational finding reproducible enough that another collaborator can rerun or reimplement it.
- Keep canon/governance status separate from evidence maturity.

- Allow the process itself to be paused, simplified, revised, or retired when it becomes the source of delay rather than learning.

Dissenting mission preserved

Dissent

A lighter alternative is to avoid a formal multi-gate protocol and use only a short falsification loop: identify one important claim, state what would prove it wrong, run a test, publish the result, and repeat. This dissent is preserved because a sophisticated review process can become another large framework that delays the work it was designed to improve. The protocol therefore includes a minimum viable mode and explicit stop rules.

3. Overview: What This Protocol Changes

OAL AI Unified has accumulated a large, diverse documentation record across research frameworks, laboratories, worldbuilding, systems design, governance concepts, operating systems, language concepts, and related development threads. That history is valuable, but scale creates a new failure mode: a team can become very good at generating documentation while losing track of which claims have been exercised, which conflicts remain unresolved, which ideas were inherited from prompts, and which files are authoritative for a given question.

The process changes the unit of progress

Old progress signal	Replacement signal
More pages / more versions	A clarified claim register showing what is source, assessment, proposal, or finding
AI agreement	Independent convergence measured only after the independent phase is preserved
A complete framework	A framework that has passed at least one falsifiable exercise
A simulated result	A model-scoped finding with assumptions, code, raw output, and replication status
A polished synthesis	A discrepancy map that preserves contradictions and unresolved evidence
A release	A reproducible bundle with source versions, manifests, hashes, and known limitations

The process does not require every topic to become a scientific experiment

Some OAL questions are creative, metaphysical, architectural, or definitional. They may not have an empirical truth condition. Those questions should not be forced into fake experiments. The protocol instead asks whether a question is testable, partially testable, operationally testable, or

not testable. A non-testable question can still be clarified, compared, versioned, and kept OPEN; what it cannot do is borrow the language of evidence from a simulation that never tested it.

4. Source Basis and Lessons From the End Cycle Cross-Review

The immediate exemplar is OAL-ECL-XR-PERPLEXITY-001 v0.2, prepared September 30, 2026 by Perplexity Computer. That document reviewed five Iteration 1 End Cycle Laboratory designs, explicitly stated that none had executed an exercise, created a 24-item discrepancy register, introduced epistemic labels, ran three pilot simulations, and issued an Iteration 2 commission that separated revision, pre-registration, execution, and translation. The supplied companion files include the Python simulation source and its JSON results.

Practices worth generalizing

- Adversarial self-review: Perplexity reviewed its own earlier work with the same severity as peer documents rather than exempting it.
- Non-consensus posture: discrepancies were given TEST, DECISION, or HYGIENE paths rather than averaged into a synthetic middle.
- Epistemic labels: SOURCE, FINDING, EVIDENCE, ASSESSMENT, PROPOSAL, and OPEN made the status of claims visible.
- Execution gate: only something actually run could produce a FINDING.
- Controls and counterfactuals: the review elevated a no-intervention branch from optional detail to a basic requirement.
- Generator/evaluator separation: the P3 exercise exposed why an author who defines hidden truth and then judges the detector can produce circular validation.
- Failure-direction symmetry: wrongful action and harmful non-action both need testing; delay and forced continuation can be harms too.
- Preserved dissent: an alternative mission was kept in the document rather than deleted after a preferred mission was selected.
- Publication-minded provenance: the code and results were explicitly treated as companion repository artifacts.

What should be strengthened beyond the exemplar

- Add a formal exact-rerun check and record whether the supplied results are byte-for-byte reproducible.
- Add implementation replication: a second collaborator should reproduce selected results using independently written code, not only the same script.
- Add dependency/environment capture: Python version, package versions, OS/container notes, command line, and file hashes should accompany every run.
- Add automated invariants/tests for simulation code so code changes cannot silently change the meaning of metrics.
- Add a prompt/source hash or immutable source reference for AI-generated judgments, because the prompt is part of the experimental apparatus.

- Add a two-axis status model so canon/ONESTAMP decisions cannot be confused with evidence maturity.
- Add an explicit registry requirement because large repositories can lose provenance even when every individual document is carefully versioned.
- Add resource-aware modes so the method is sustainable without requiring paid computer work on a monthly schedule.

5. Independent Feedback, Criticism, Questions, and Concerns

Feedback

The strongest contribution of the Perplexity exercise is that it changed the collaboration norm from "we designed an exercise" to "show me the run." That is a meaningful standard to carry forward. The three simulations do not establish the End Cycle Laboratory's substantive theory, but they demonstrate what the next phase of OAL work can look like: assumptions are visible, outputs exist, code can be rerun, and a result can contradict a preferred rule.

Criticism

The pilot remains a one-author toy-model study. Its parameters encode many of the outcomes it later reveals. For example, P2 explicitly makes an independent reviewer less biased than the initiating operator; P3 explicitly makes the external signals identical by construction. Those are legitimate pilot choices because the document labels them as author-chosen assumptions, but the results should be treated as demonstrations of test mechanics, not validation of the underlying governance or ontology. A second implementation with independently chosen assumptions is therefore more informative than merely increasing the number of Monte Carlo runs.

Concerns

- False independence. If all collaborators receive the same source packet, prompt structure, and prior synthesis, convergence can be prompt inheritance rather than discovery. Preserve a blind independent phase before exchange.
- Full-team pass contamination. Once one collaborator sees another's criticism, later reviews are no longer independent. Preserve each original review and label the cross-review stage separately.
- Framework bureaucracy. The review process can become another large OAL structure. The process needs a minimum viable mode, time budget, and stop rule.
- Version explosion. With a repository at OAL's scale, filenames alone will eventually become insufficient. A machine-readable registry should track topic, document ID, version, status, supersedes, depends-on, execution state, and artifact hashes.
- Overclaiming simulation evidence. More runs do not repair author-chosen assumptions. Parameter sweeps, reimplementations, and external data are different evidence upgrades.
- Unreviewed code. AI-generated code can be internally consistent and still implement the wrong test. The test design and the implementation need separate review lines.

Questions for the team

1. What is the minimum evidence required before an OAL framework may describe itself as "tested"?
2. Should every major topic have one designated reproducibility bundle, or should each AI maintain its own bundle until cross-review?
3. Which decisions are reserved exclusively for the Architect, and which should remain unresolved until testing changes the evidence?
4. How much independence should be preserved by withholding peer documents during the first pass?
5. What is the retirement rule for an OAL framework that repeatedly fails exercises or no longer answers a live project question?
6. Which topics are intentionally non-testable and should remain conceptual rather than being forced into simulations?

6. Core Design Principles

ID	Principle	Operational meaning
P-01	Evidence before expansion	When a claim can be tested, attempt a small falsifiable test before adding more architecture around it.
P-02	Specification is not evidence	A prompt, framework, test plan, scorecard, or scenario is not a finding until it is executed.
P-03	Disagreement is data	Conflicts between collaborators are preserved, classified, and routed; they are not averaged away.
P-04	Independence is declared, not assumed	Every reviewer lists the inputs it saw. A reviewer cannot claim independence from material it read.
P-05	Baselines are mandatory when meaningful	Ordinary change, no-action, or no-intervention branches prevent the test from labeling every difference as the target phenomenon.
P-06	Test both error directions	Action can be wrong; inaction, delay, forced continuation, over-preservation, or indefinite review can also be wrong.
P-07	Separate roles that can bias one another	Where possible, split source author, world/generator author, evaluator, red-team reviewer, and final synthesizer.
P-08	Raw artifacts outrank summaries for reproducibility	Publish code, prompts, parameters, raw results, environment, and hashes alongside prose conclusions.

ID	Principle	Operational meaning
P-09	Unknown is a valid result	INDETERMINATE, NOT RUN, UNSUPPORTED, and OPEN must remain available outputs.
P-10	The protocol can fail too	If the process becomes expensive bureaucracy, it must be simplified or paused. No review system is exempt from adversarial review.

7. Status, Evidence Maturity, and Epistemic Labels

Two-axis status model

Project authority and evidence maturity should be tracked separately. A worldbuilding decision may be canon because the Architect declares it so even if it is not empirically testable. Conversely, a simulation result can be reproducible evidence about a model while remaining entirely non-canon. Combining those two dimensions invites confusion.

Axis	Allowed states	Question answered
A. Project / governance status	DRAFT; PROPOSED; CANON (if applicable); ONESTAMP-RATIFIED / NOT RATIFIED; SUPERSEDED; RETIRED	What authority does this artifact have inside the project?
B. Evidence maturity	SPECIFICATION; PREREGISTERED; PILOT-EXECUTED; REPLICATED; CROSS-IMPLEMENTATION-REPLICATED; EXTERNALLY-VALIDATED; INDETERMINATE	What evidence exists for the claims made by this artifact?

Epistemic labels

Label	Meaning
SOURCE	Directly stated in a named source artifact. Quote or paraphrase with provenance.
FINDING	Produced by an executed test or run. Always scoped to the tested model/data.
EVIDENCE	External or independent material relevant to a claim. Does not automatically alter canon.
ASSESSMENT	Reviewer interpretation or critique.
PROPOSAL	New design, rule, model, definition, or process offered for consideration.
OPEN	Not resolved by the current corpus or test.
DISSENT	A deliberately preserved alternative interpretation or recommendation.

Label	Meaning
REPLICATION	A finding specifically produced by rerunning or independently reimplementing a prior test.
Rule Only executed work produces FINDING or REPLICATION labels. A predicted outcome, expected result, or cross-model agreement remains ASSESSMENT or PROPOSAL until a run exists.	

8. When to Run the Process: Milestone Triggers and Resource Modes

Use event triggers, not a monthly obligation

- Before a major framework is called complete, operational, executable, or validated.
- Before a major artifact is considered for canon or ONESTAMP when the decision depends on claims that are testable.
- After a major redesign changes the assumptions, architecture, or test battery.
- When two or more collaborator versions materially conflict on definitions, authority, test design, or conclusions.
- When a simulation result is about to be generalized beyond its original toy model.
- When a topic has accumulated enough documents that provenance or contradictions are becoming difficult to track.
- After a failed or contradictory replication.
- Before a real-world translation, publication, demonstration, or external claim relies on the work.

Resource-aware operating modes

Mode	Cost profile	Required outputs	What it may claim
R0 - Documentation audit	\$0 / routine	Corpus register, independent review, discrepancy register, unperformed-task list	Specification/assessment only; no findings
R1 - Lightweight executable	\$0-low	At least one small test, baseline if applicable, raw output, limitations, rerun command	Pilot findings about the tested model
R2 - Cross-AI replication	Existing subscriptions / moderate effort	Independent reviewers, blind relabeling or second implementation, discrepancy comparison	Replicated or contradictory model findings

Mode	Cost profile	Required outputs	What it may claim
R3 - Computer-assisted deep pass	Paid computer/cloud when justified	Larger parameter sweeps, automated batch review, external datasets, stronger provenance bundle	Higher-confidence model or translation evidence, still scoped to assumptions
R4 - External / human / operational	Potentially high cost; ethics/legal review as applicable	Independent data, approved participant studies, operational evidence	Claims tied to the actual external population/environment studied

Budget principle

Reserve expensive computer runs for claims that survive R0 and R1. Do not pay to scale a test whose assumptions have not yet survived adversarial review.

9. The Full Adversarial Review and Validation Lifecycle

The lifecycle below is the default full process. Gates may be combined for small topics, but their functions should remain visible so that a reader can tell whether a document is specifying, testing, replicating, or translating.

Gate	Name	Required outcome	Stop condition
G0	Commission / corpus freeze	Frozen source register, scope, status, missing-input list, version map	Do not review a moving corpus without recording the cut-off.
G1	Independent adversarial review	One review per collaborator; contribution, defects, assumptions, omissions, testable claims	Do not exchange peer reviews until independent versions are saved.
G2	Registers	Discrepancy register, claim ledger, open questions, unperformed tasks, provenance issues	No issue disappears merely because a synthesis is written.
G3	Test conversion / pre-registration	Hypotheses, falsifiers, parameters, controls, metrics, stop rules, role separation	No FINDING before preregistered run unless explicitly labeled exploratory.
G4	Execute	Code/prompts, raw data, failures, logs, figures, run manifest	Every unrun/failed/indeterminate exercise is reported.
G5	Replicate / blind	Exact rerun + at least one stronger replication for important claims	No broad claim from a single-author self-confirming setup.
G6	Evidence synthesis	Result matrix, contradiction register, limitation map, dissent	No single score or consensus verdict may erase dimensions.
G7	Iteration commission	Keep/change/reject ledger tied to evidence; next tests commissioned	No change without its reason and provenance.

Gate	Name	Required outcome	Stop condition
G8	Publication / archive	Versioned release bundle, hashes, environment, manifest, supersedes/dependencies	No release called reproducible if artifacts are missing.

10. Gate 0 - Commission, Scope, and Corpus Freeze

Required source register

Field	Required content
Topic / project	Exact topic name and organizational prefix.
Cut-off date/time	The last source version included in the review.
Artifact ID	Document ID and version; preserve original filename.
Preparer	Exact preparer attribution as published.
Status	Draft/proposed/canon/ratified/superseded/etc.
Role in corpus	Source, prior review, implementation, results, external evidence, or translation.
Read by reviewer?	Yes/no; this supports the independence declaration.
Hash / immutable ref	SHA-256, commit, or stable repository reference when available.
Missing / secondhand	Explicitly identify sources discussed but not available firsthand.

Corpus freeze rules

- Never silently replace a missing source with another collaborator's summary of it.
- If a later document arrives, either open a new review version or add it as a labeled supplemental source.
- Preserve the exact prompts used to generate the reviewed artifacts when available.
- If source files contain internal contradictions, record them; do not repair the source before review.
- For GitHub, prefer commit IDs and hashes over mutable branch names when identifying the reviewed state.

11. Gate 1 - Independent Adversarial Review

Each collaborator should review the corpus independently before reading the other reviewers' new analyses. This is the phase that protects genuine divergence. The review should be severe enough to criticize the reviewer's own prior work.

Seven mandatory questions for every source artifact

7. What does this artifact contribute that the rest of the corpus does not?
8. What does it do well enough to preserve?
9. Where is it internally inconsistent or ambiguous?
10. Where does it assume the conclusion it claims to test?
11. Which claims are testable, and which remain conceptual or definitional?
12. What did it say it would execute that it never actually executed?
13. What should the next iteration keep, change, reject, quarantine, or test?

Mandatory self-review rule

Self-review

If the reviewer has a prior contribution in the corpus, review it first or in a separately labeled section. Identify at least one concrete weakness, one untested assumption, and one change that would make the prior contribution more falsifiable.

12. Gate 2 - Discrepancy, Contradiction, and Open-Work Registers

Discrepancy register

The discrepancy register is the center of the process. It turns cross-review from prose comparison into a map of specific conflicts. Each discrepancy receives a resolution path; none is solved merely because most collaborators agree.

Field	Description
ID	Stable topic-prefixed identifier, e.g., TEMP-D-001.
Issue	Short neutral statement of the conflict.
Positions	Each distinct source position with attribution.
Why it matters	What downstream test, architecture, or interpretation changes depending on the answer.
Resolution route	TEST, DECISION, HYGIENE, RESEARCH, BLOCKED, or RETIRE.
Owner / next action	Who or what must act next.
Status	OPEN, IN TEST, RESOLVED BY EVIDENCE, RESOLVED BY DECISION, DEFERRED, SUPERSEDED.
Evidence links	Run IDs, documents, or commits that bear on the discrepancy.

Additional registers

- Claim ledger: Every important claim, its source, testability, falsifier, evidence maturity, and current status.
- Unperformed-task register: Tests, analyses, comparisons, or promised artifacts that the documentation says should exist but do not.
- Open-question register: Questions genuinely unresolved by available sources or runs.
- Provenance-error register: Misattribution, missing source, version mismatch, copied hypothesis IDs, stale references, or secondhand claims.
- Assumption register: Model assumptions that materially determine outputs; includes who chose them and whether they were varied.
- Dissent register: Alternative missions, interpretations, or recommendations intentionally preserved across iterations.

13. Gate 3 - Test Conversion and Pre-registration

Convert prose into a test card

Test-card field	Required entry
Question	What exact uncertainty is the test trying to reduce?
Target claim	The claim being challenged, with source ID.
Hypothesis	A concise statement that can be false.
Falsifier / kill condition	What observable outcome would count against the claim?
Baseline / control	No-intervention, ordinary-change, alternative rule, or other reference branch.
Alternative hypotheses	At least one plausible competing explanation.
Inputs	Source scenarios, data, fixtures, prompts, or initial state.
Parameters	Values fixed in advance and the reason for them.
Parameter sweep	Which assumptions will be varied and over what range.
Metrics	Separate dimensions; avoid a single composite score unless independently justified.
Roles	Generator, evaluator, red-team, executor, synthesizer.
Run plan	Runs, seeds, branches, time horizon, stopping criteria.
Outcome classes	PASS, FAIL, MIXED, INDETERMINATE, NOT RUN, ERROR.
Artifacts	Expected code, raw output, logs, figures, and manifest.
Safety / ethics	Any real-world risk, human participant, privacy, or misuse concern.

Pre-registration rule

Commit or otherwise freeze the test card before the result is known. Exploratory tests are allowed, but they must be labeled EXPLORATORY and may not be presented as if the hypothesis and thresholds were fixed in advance.

14. Gate 4 - Execution Standards

Minimum executable standard

- One command, notebook, script, structured prompt, or reproducible procedure that another reviewer can execute.
- A baseline or comparison branch where the phenomenon could otherwise be confused with ordinary behavior.
- Raw output preserved before interpretation.
- All failures, errors, and unrun cases listed - not deleted from the final report.
- Assumptions stated next to results, not buried only in an appendix.
- Run identifier, date, environment, source versions, and random seed where relevant.
- Separate metrics for distinct concerns; do not use one number to merge process, outcome, legitimacy, persistence, cost, or other independent dimensions.
- A model-scope statement: what the run shows about the model, and what it does not show about the world.

Code-specific requirements

Requirement	Purpose
Deterministic exact-rerun path	Shows that the published raw result can be regenerated under the same conditions.
Seed sweep	Shows whether a stochastic headline depends on one random draw.
Parameter sweep	Shows where a conclusion changes when assumptions change.
Assertions / invariants	Catches implementation drift and impossible states.
Environment lock or manifest	Records language/package versions.
Independent code review	Checks whether the implementation matches the pre-registered test.
Second implementation for important claims	Reduces the risk that both the result and bug are inherited from the same codebase.

AI-judgment-specific requirements

- Preserve the exact prompt and relevant context packet.

- Use at least two independent raters for important categorical judgments.
- Report agreement and disagreement; do not merge the answers before measuring reliability.
- Use blinded relabeling when scenario names or source identities could cue the intended answer.
- Where a generator knows hidden ground truth, the evaluator should not have access to that information.
- Re-run selected cases with materially different prompts to estimate prompt sensitivity.

15. Gate 5 - Replication, Blinding, and Independent Judgment

Replication ladder

Level	Replication type	What it checks
L0	Exact rerun	Same code, same seed, same environment or compatible environment; verifies artifact reproducibility.
L1	Seed replication	Same code, different random seeds; checks stochastic stability.
L2	Parameter replication	Same implementation, altered assumptions; checks boundary conditions and conclusion flips.
L3	Independent reimplementation	New code/procedure from the same preregistration; checks implementation dependence.
L4	Cross-AI replication	Different collaborator independently designs/evaluates the same core question; checks model/prompt dependence.
L5	External validation	Independent data, real systems, or approved participant study; checks transfer beyond the toy model.

Not every topic must reach L5. The important point is to stop describing L0 or L1 as if it were L5. Evidence maturity must match the strongest completed replication level.

Role separation pattern

Role	Sees	Must not control
Source curator	Corpus and provenance	Final interpretation
Generator / scenario author	Test design, hidden truth if required	Evaluator scoring
Executor	Pre-registered implementation	Post-hoc thresholds
Evaluator	Permitted observations only	Hidden labels / intended answer

Role	Sees	Must not control
Red-team reviewer	Method, code, results	Ability to rewrite raw data
Synthesizer	All preserved artifacts after independent stage	Deletion of dissent or failed runs

16. Gate 6 - Evidence Synthesis Without Consensus Laundering

The synthesis does not ask "what does the team agree on?" as its first question. It asks what was tested, what happened, where results reproduce, where they diverge, which assumptions explain the divergence, and what remains unknowable from the current design.

Required result matrix

Claim / test	Reviewer or run	Outcome	Evidence maturity	Contradiction?	Next action
Example claim A	AI-1 / RUN-001	PASS	Pilot-executed	AI-2 failed	Parameter sweep
Example claim A	AI-2 / RUN-002	FAIL	Independent reimplementation	Yes	Inspect assumption mismatch
Example claim B	AI-3	NOT RUN	Specification	N/A	Commission test
Example claim C	AI-4	INDETERMINATE	Pilot-executed	No	Improve instrument

Synthesis rules

- Never convert a 4-to-2 majority into proof.
- Where two implementations disagree, preserve both results until the cause is understood.
- If a conclusion changes under plausible parameters, report the boundary, not a single headline.
- If a metric requires subjective coding, report inter-rater disagreement as part of the result.
- If a result only follows because the test designer encoded the separation, state that explicitly.
- If no test can resolve a question, keep it OPEN rather than inventing a pseudo-measurement.

17. Gate 7 - Iteration Commission and Change Control

Every iteration begins with a change ledger

Change type	Required statement
KEEP	What survives and what evidence or design reason supports keeping it.
CHANGE	Exact change, the discrepancy/finding that motivates it, and expected effect.

Change type	Required statement
REJECT	What is removed and why.
QUARANTINE	What remains archived but may not be used as an active premise until a dependency is resolved.
ADD	New element and whether it is a proposal or response to an executed finding.
DEFER	Open item, dependency, and trigger for revisiting it.

A new version must not rewrite the prior record. It should supersede or amend it through explicit version links while preserving the original artifact for GitHub history.

Iteration stop rules

- Stop adding structure when the next unanswered question can be addressed by an existing instrument.
- Stop scaling a simulation when the limiting uncertainty is an assumption rather than sampling error.
- Stop a review cycle when new reviewers are only repeating already registered discrepancies.
- Pause the protocol if review overhead exceeds the expected learning value of the next decision.
- Retire a framework when its core purpose disappears, repeated tests defeat its premise, or a simpler replacement covers the same function better.

18. Gate 8 - Publication, GitHub Packaging, and Reproducibility

Minimum release bundle

Artifact	Minimum content
README / release note	Topic, question, status, evidence maturity, what changed, how to reproduce.
Primary DOCX/PDF	Human-readable review or report.
Source register	Exact reviewed corpus and missing-input notes.
Pre-registration	Frozen test cards / hypotheses / falsifiers.
Code or procedure	Executable implementation or exact structured procedure.
Raw results	JSON/CSV/text/logs before narrative interpretation.
Figures / derived tables	Generated from raw outputs, with script or notebook.
Run manifest	Environment, versions, seeds, command, hashes.
Discrepancy register	Open/resolved conflict state.
Change ledger	Keep/change/reject/quarantine/add/defer decisions.

Artifact	Minimum content
Checksums	SHA-256 or equivalent for release artifacts.

Machine-readable registry recommendation

For a repository with a large document count, add a registry file (CSV, JSON, YAML, or database export) as a first-class artifact. The registry does not replace the documents. It makes them queryable.

Registry field	Example
artifact_id	OAL-AIU-ARVIP-CHATGPT-001
version	0.1
topic	adversarial-review-protocol
artifact_type	protocol
preparer	ChatGPT (OpenAI)
date	2026-10-01
governance_status	PROPOSED
evidence_maturity	SPECIFICATION
supersedes	null
depends_on	OAL-ECL-XR-PERPLEXITY-001 v0.2
execution_state	not-applicable / protocol
path	relative GitHub path
sha256	release checksum

19. Minimum Common Templates

The full forms appear in Appendix A. At minimum, every cross-topic review cycle should use the following artifacts.

- Form A - Source / independence declaration.
- Form B - Independent adversarial review worksheet.
- Form C - Discrepancy register.
- Form D - Claim and evidence ledger.
- Form E - Pre-registration / test card.
- Form F - Run manifest.
- Form G - Exercise result report.
- Form H - Replication record.
- Form I - Iteration change ledger.
- Form J - GitHub release checklist.

20. Copy-Ready Cross-Topic Master Commission Prompt

Use case

This prompt is designed to be sent independently to ChatGPT, Claude, Gemini, Copilot, Grok, Perplexity, or another collaborator. For a genuinely independent first pass, do not attach the new reviews from the other collaborators until each independent version has been saved.

ROLE AND POSTURE

You are an independent adversarial reviewer and validation designer working on [PROJECT / TOPIC] for Omniversal Architect Labs. Your job is not to seek consensus, praise the corpus, or reconcile every difference. Your job is to identify what is supported, what is assumed, what conflicts, what is missing, what has not been executed, and what can be tested next. You may conclude that a claim, framework, exercise, or institution should be revised, narrowed, quarantined, or retired.

SOURCE DECLARATION

List every source you received by exact title/ID/version and state whether it was read directly or only described secondhand. Do not claim independence from a source you read. Identify missing sources explicitly.

EPISTEMIC LABELS

Use SOURCE, FINDING, EVIDENCE, ASSESSMENT, PROPOSAL, OPEN, and DISSENT. Only executed work may be labeled FINDING. If you run a reproduction of prior work, label it REPLICATION.

INDEPENDENT REVIEW

Review each source separately. For each: (1) unique contribution, (2) strongest part, (3) internal inconsistency, (4) hidden assumption or circularity, (5) claims that can be tested, (6) promised or implied tasks not performed, and (7) what the next iteration should keep/change/reject/quarantine/test. If you authored a prior source, review your own source with equal or greater severity.

REGISTERS

Create: a discrepancy register; claim/evidence ledger; unperformed-task register; open-question register; and provenance-error register. Do not resolve discrepancies by averaging. Route each discrepancy as TEST, DECISION, HYGIENE, RESEARCH, BLOCKED, or RETIRE.

TESTABILITY

Classify important questions as TESTABLE, PARTIALLY TESTABLE, OPERATIONALLY TESTABLE, or NON-TESTABLE. Do not invent pseudo-tests for non-testable metaphysical or purely creative questions.

PRE-REGISTRATION

For the most important testable claims, define the hypothesis, falsifier, control/baseline, competing explanation, parameters, parameter sweep, metrics, roles, stop rule, and expected artifacts before seeing the result. Label any post-hoc test EXPLORATORY.

EXECUTION

Run at least one small exercise if tools and scope permit. Prefer the smallest test that could prove the claim wrong. Preserve raw output, failures, code/procedure, seed, environment, and run instructions. If execution is not possible, say NOT RUN and provide the exact executable plan instead of fabricating a result.

REPLICATION

For important computational results, distinguish exact rerun, seed replication, parameter replication, independent reimplementations, cross-AI replication, and external validation. Do not call a same-code rerun independent validation.

BLINDING / ROLE SEPARATION

Where an author can encode the answer into the test, separate generator, evaluator, and red-team roles. Use blinded relabeling for subjective judgments. Report inter-rater agreement and disagreement.

SYNTHESIS

Produce a result matrix that preserves contradictions. Do not use a majority vote or one composite score to erase independent dimensions. State where conclusions flip under plausible assumptions.

ITERATION COMMISSION

End with KEEP / CHANGE / REJECT / QUARANTINE / ADD / DEFER decisions tied to specific findings or discrepancies, then commission the next smallest useful tests.

PUBLICATION

Prepare the work for GitHub: title, document ID, version, date, preparer, project status, evidence maturity, source register, changelog, raw artifacts, code/procedure, run manifest, and hashes. Never alter prior versions to make the history look cleaner.

REQUIRED DOCUMENT STRUCTURE

Title page; mission/directive; executive summary; overview; source corpus; method; independent source reviews; discrepancy register; testability analysis; executed tests or NOT RUN plan; results; limitations; questions; iteration commission; reproducibility record; appendices.

CLOSING RULE

Run the smallest test that could prove you wrong before building the largest structure that could make you look right. Report what failed, what could not be distinguished, what remains open, and which assumptions paid for the result.

21. Quality Gates and Automatic Review Failures

A review/report should fail the protocol quality gate if any of the following occurs without explicit limitation and correction:

- Predicted lessons or expected outcomes are presented as FINDINGS.
- A framework is called tested even though no exercise was run.
- Raw outputs are missing for a computational headline result.

- Parameters, thresholds, or scoring rules are changed after the run without being identified as post-hoc.
- The same author defines hidden ground truth and evaluates success, then calls the result independent.
- No meaningful baseline/control exists where ordinary change could mimic the target phenomenon.
- Only one failure direction is tested when the opposite error could also cause harm or invalidity.
- A single composite score hides distinct dimensions or trade-offs.
- Failed, error, indeterminate, or unrun exercises are omitted.
- Independence is claimed without listing the inputs the reviewer saw.
- Secondhand descriptions are represented as if they were the missing source's own words.
- Results cannot be rerun because code, prompts, environment, or data are absent.
- A same-code rerun is described as an independent implementation.
- A toy-model result is generalized to the real world without a Translation / external-evidence stage.
- Peer disagreement is erased through majority vote, averaging, or prose synthesis that omits the contradiction.
- The review process has no mechanism for pausing or retiring itself.

22. Adversarial Failure-Mode Catalogue

ID	Failure mode	Diagnostic
FM-01 Consensus laundering	Multiple AIs repeat a premise inherited from the same prompt, and repetition is mistaken for independent evidence.	
FM-02 Prompt inheritance	The commission prescribes the answer shape so strongly that apparently independent outputs converge structurally.	
FM-03 Specification inflation	More definitions, offices, stages, or instruments are added before any existing instrument is run.	
FM-04 Stage inflation	A document labels itself as a later stage even though required execution evidence is absent.	
FM-05 Simulation circularity	The model author builds assumptions that guarantee the result and then cites the run as validation.	
FM-06 Metric laundering	A composite score conceals trade-offs or embeds value judgments as arithmetic.	
FM-07 Baseline blindness	Ordinary variation is misclassified because there is no control branch.	

ID	Failure mode	Diagnostic
FM-08 One-direction testing	Only the feared action is tested; delay, forced continuation, over-preservation, or inaction are ignored.	
FM-09 Rater monoculture	One model supplies every categorical judgment, so a table of numbers is actually one opinion repeated.	
FM-10 Provenance drift	Preparer, version, source ID, or hypothesis labels mutate as documents cite one another.	
FM-11 Citation laundering	A collaborator cites another collaborator's statement of a source instead of the source itself.	
FM-12 Version shadowing	A newer file silently replaces an older artifact without an explicit supersedes record.	
FM-13 Result overreach	A finding about a toy model is written as a claim about reality, canon, or all possible systems.	
FM-14 Compute theater	More runs or larger models are purchased even though the dominant uncertainty is conceptual assumptions.	
FM-15 Bureaucratic capture	The review system becomes harder to pause than the work it reviews.	
FM-16 Archive without index	Thousands of well-versioned files become practically unsearchable because no registry links them.	
FM-17 Hidden non-execution	A framework contains many exercises, but status is never tracked, so readers assume more was run than actually was.	
FM-18 Post-hoc certainty	Thresholds and success criteria are chosen after results are visible, then presented as if pre-specified.	
FM-19 Missing negative results	Failed or contradictory tests are excluded from the release bundle.	
FM-20 Translation collapse	External evidence is used to rewrite a fictional source scene or canon instead of being kept in a separate translation track.	

23. How to Apply the Protocol to Different OAL Topics

A. Laboratory / research framework

Example: temporal integrity, temporal aliasing, sensing, end cycles, or a generalized superposition framework. Convert definitions into misclassification tests, invariants, counterfactual branches, synthetic cases, and blind diagnosis tasks. Treat simulations as model evidence until independent implementation or external data exists.

B. Governance or institutional design inside the OAL universe

Do not begin by asking which office should exist. First test the failure modes the office is supposed to address. Compare at least one design with no new institution. Track action error, inaction/delay error, bypass behavior, self-entrenchment, and who bears transition costs. Keep in-world authority decisions separate from empirical claims about the simulation.

C. Language / operating-system / architecture design

For CoreLang, 1script, -1script, VoidScript, Architect OS, or related system designs, execution means parsers, type rules, state-machine traces, conformance fixtures, property tests, reference programs, failure cases, and differential implementations - not only syntax documents. A minimal executable grammar or state machine can often expose contradictions earlier than another design dossier.

D. Pure worldbuilding / narrative concept

Not every narrative concept requires simulation. Instead, test internal consistency: continuity rules, chronology, identity constraints, state transitions, and whether later documents silently assign meaning to intentionally open source material. Label interpretive readings as PROPOSAL and preserve multiple readings when canon has not been fixed.

24. Case Study: End Cycle Laboratory Reproducibility Check

Supplied artifacts checked

Artifact	Role	SHA-256
End Cycle Laboratory Adversarial Cross-Review v0.2 (DOCX)	Primary cross-review / Iteration 2 commission	See source register; document reviewed in full via supplied file.
ECL Iteration 2 pilot simulations (pilot.py)	Simulation implementation	47d70d1b414cc626d5be3ba2a5221cc20f901c49b5c03b377133a630a921f812
ECL pilot results (results.json)	Published raw aggregate results	7813a3a555f6ee73792e443a449bc0dbe11e86f86034db21df5bf2f38ddb5f84

Independent exact-rerun result

On October 1, 2026, ChatGPT reran the supplied pilot.py without changing its computational logic. The expected output directory was created so the script could write its files. The rerun produced results.json that matched the supplied results.json byte-for-byte and had the same SHA-256 hash. The environment used Python 3.13.5, NumPy 2.3.5, Matplotlib 3.10.8, on Linux 6.18.44 x86_64. This is a successful L0 exact-rerun replication. It demonstrates reproducibility of the supplied implementation under this environment; it does not validate the author-chosen model assumptions.

Selected result checks

Pilot	Published result pattern	Independent assessment
P1 - field trajectories	Immediate, staged, and adaptive trajectories all reached the model's completion condition, but persistence differed sharply. Immediate lost 98% of mobile fauna and 100% of sessile entities on average; adaptive lost 2% mobile and 35.2% sessile. The no-end baseline still produced 60.5% dormancy.	Strong demonstration of why completion alone is an insufficient metric and why a baseline matters. Still model-scoped because loss/dormancy probabilities are author-defined.
P2 - governance	At urgent share 0.10 and review delay 8, single authority wrongful rate was 0.211; split authority 0.011. Urgent harm rose from 0.151 to 0.546. Refusing upset petitioners lowered wrongful rate to 0.002 but raised urgent harm to 0.865 and blocked 0.609 of warranted petitions.	Strong demonstration of a trade-off and why one-sided metrics can mislead. Requires independent assumptions/implementation before becoming a broader governance conclusion.
P3 - indistinguishability	External-only accuracy stayed around 0.50 because external signals were constructed identically. An internal probe improved as the author-defined separation increased, reaching 0.84 at separation 0.30.	A useful demonstration of circularity: detectability follows the separability the generator encoded. The next step is split authorship, not simply more runs.
Replication conclusion The pilot passes an exact-rerun reproducibility check. Its next meaningful evidence upgrade is not a larger same-code Monte Carlo run. It is an independent reimplement and/or generator-evaluator split with independently chosen assumptions.		

25. Questions Reserved for the Architect

- Should this protocol apply only to OAL AI Unified, or also be adopted as a shared review standard for One Industries and Lewis Corp?
- What governance/status terms are authoritative project-wide (for example, PROPOSED, CANON, ONESTAMP-RATIFIED, RETIRED), and which are topic-specific?
- Do you want a central machine-readable artifact registry added to the GitHub archive, or should each topic maintain its own registry?
- Which milestones should automatically trigger a full cross-review: pre-ONESTAMP, pre-canon, major version, failed replication, or another threshold?
- How blind should the independent pass be? Should reviewers see only the primary source packet, or also their own prior version?
- When a discrepancy is fundamentally a worldbuilding choice rather than a testable question, should the protocol route it directly to Architect Decision without further simulation?
- What minimum replication level is required before an OAL document may use the word "validated"?

21. How should archival supersession work for the existing large corpus without renaming or breaking Git history?

26. Recommended Adoption Path

Phase 1 - Trial the protocol on one existing topic

Use a topic that already has multiple AI versions and at least one executable claim. Do not start by retrofitting the entire archive. The goal is to test whether the protocol produces better decisions and clearer provenance than the current workflow.

Phase 2 - Ask all six collaborators for independent versions

Send each collaborator the same source packet and the high-level request to design a replicable adversarial review protocol. Preserve each independent result before distributing peer versions. The first comparison should focus on differences in process, not on forcing one master version immediately.

Phase 3 - Cross-review the protocol itself

Apply the protocol to the six protocol documents. This is an important self-test: if the protocol cannot manage conflicting designs of itself, it should not be imposed on other OAL topics.

Phase 4 - Create the registry and lightweight templates

After the process survives the self-test, create a central registry and standardized small forms. Keep them lightweight enough that ordinary documentation does not become administrative work.

Phase 5 - Trigger deep reviews only at milestones

Use R0/R1 routinely and reserve R2/R3 for important milestones. This makes the method sustainable even when paid Computer runs are occasional rather than monthly.

Closing position

The team should keep generating ambitious frameworks, but the archive should increasingly show two different kinds of artifacts: specifications that say what we think might work, and evidence packages that show what happened when we tried to make those ideas fail. The distinction should be visible at a glance.

Appendix A. Reusable Forms and Tables

Form A - Independence declaration

Field	Entry
Reviewer	
Topic / cycle	
Sources read directly	
Sources described secondhand	
Peer reviews seen before independent pass	Yes / No; list if yes
Prior contribution by this reviewer	
Known conflicts / constraints	
Independence claim	I claim independence only from materials not read or supplied.

Form B - Independent adversarial review worksheet

Field	Entry
Artifact ID / version	
Unique contribution	
What should be preserved	
Internal contradictions	
Hidden assumptions / circularity	
Testable claims	
Non-testable claims	
Tasks promised but not executed	
Provenance problems	
Keep / change / reject / quarantine / test	

Form C - Discrepancy register row

ID	Issue	Positions	Why it matters	Route	Status	Evidence / owner
				TEST / DECISION / HYGIENE / RESEARCH / BLOCKED / RETIRE		

Form D - Claim and evidence ledger row

Claim ID	Claim	Source	Testability	Falsifier	Evidence maturity	Current state
			T / PT / OT / NT		SPEC / PILOT / REPL / EXT	

Form E - Pre-registration / test card**Question:** _____**Target claim:** _____**Hypothesis:** _____**Falsifier:** _____**Baseline/control:** _____**Competing explanation:** _____**Inputs:** _____**Fixed parameters:** _____**Parameter sweep:** _____**Metrics:** _____**Role separation:** _____**Run count / seeds:** _____**Stop rule:** _____**Outcome classes:** _____**Expected artifacts:** _____**Safety / ethics:** _____**Pre-registration commit/hash:** _____

Form F - Exercise result report

Field	Required content
Run ID	
Pre-registration ref	
Outcome	PASS / FAIL / MIXED / INDETERMINATE / NOT RUN / ERROR
Raw result location	
Key numbers / observations	
Assumptions that mattered	
Where conclusion flips	
Unexpected behavior	
Limitations	
Next replication level	

Form G - Iteration change ledger

Item	KEEP / CHANGE / REJECT / QUARANTINE / ADD / DEFER	Reason / evidence	Affected artifacts	Next action

Appendix B. Proposed GitHub Directory Structure

```

topic-slug/
  README.md
  registry.yaml
source/
  corpus-register.csv
  prompts/
  reviews/
  independent/
  cross-review/
  registers/
  discrepancies.csv
  claims.csv
  open-questions.md
  unperformed-tasks.md
prereg/
  test-cards/
execution/

```

src/
 tests/
 raw/
 derived/
 figures/
 env/
 replication/
 exact-rerun/
 reimplementation/
 cross-ai/
 iterations/
 change-ledgers/
 commissions/
 releases/
 manifests/
 checksums/

This is a proposal, not a requirement to reorganize the existing archive. For mature repositories, begin with a registry that points to current locations rather than moving historical files and breaking links.

Appendix C. Proposed Artifact and ID Codes

Code	Artifact type
SRC	Source / foundational record
IR	Independent review
XR	Cross-review
DR	Discrepancy register
CL	Claim ledger
PR	Pre-registration
EX	Execution report
RR	Replication report
IC	Iteration commission
CH	Change ledger
RM	Run manifest
REL	Release manifest

Proposed generic pattern: {ORG}-{TOPIC}-{TYPE}-{PREPARER}-{NNN}_v{MAJOR.MINOR}. Existing OAL IDs should not be retroactively renamed solely to satisfy this proposal. Instead, create a mapping registry and use a consistent pattern prospectively after Architect approval.

Appendix D. Run Manifest / Reproducibility Record

Field	Meaning
run_id	Unique run identifier
date_time_utc	Execution timestamp
operator	Human/AI/tool executing the run
preparer	Author of code/procedure
source_commit	Git commit or immutable source reference
code_sha256	Hash of executable artifact
preregistration_ref	Commit/hash of test card
input_hashes	Hashes of data/prompts/fixtures
python_or_runtime	Language/runtime version
dependencies	Package versions or lockfile
platform	OS/container/hardware notes
command	Exact command or execution procedure
seed	Random seed(s) if applicable
parameters	Fixed and swept parameters
raw_output_path	Location of raw results
raw_output_sha256	Hash of raw result
exit_status	Success/error
known_warnings	Warnings or deviations
replication_level	L0-L5
scope_statement	What the result does and does not support

Appendix E. Source Register and Provenance Notes

Source	Use in this protocol
OAL-ECL-XR-PERPLEXITY-001 v0.2, End Cycle Laboratory: Adversarial Cross-Review, Mission Statement, and Iteration 2 Commission, September 30, 2026, prepared by Perplexity Computer	Primary methodological exemplar. Used for the non-consensus review posture, discrepancy-register model, epistemic labels, execution emphasis, pre-registration/controls/reliability requirements, and the Iteration 2 gate concept.
ECL Iteration 2 pilot simulations (pilot.py), supplied with the cross-review	Reviewed and exact-rerun. Used as a case study in reproducibility and model-scope limitations.
ECL pilot results (results.json), supplied with the cross-review	Compared against the rerun output; matched byte-for-byte in the October 1, 2026 exact rerun.

Source	Use in this protocol
User commission / project direction, October 1, 2026	Defines the cross-topic goal: make the Perplexity process replicable for OAL AI Unified content, preserve criticism rather than consensus, support independent AI versions and later full-team cross-review, and publish PDF/DOCX versions to GitHub.

Version history

Version	Date	Preparer	Change
v0.1	2026-10-01	ChatGPT (OpenAI)	Initial independent protocol render. Adds milestone-driven resource modes, two-axis status, exact-rerun verification, replication ladder, machine-readable registry recommendation, and reusable cross-topic templates.

END OF DOCUMENT